Faculty of Computing

Riphah International University — I-14 Main Campus

LAB 07

Tic Tac Toe — AI with Minimax & Evaluation Function

Artificial Intelligence | CS 3703

Student Name:	Mohammad Sami	**Student ID:**	53199
Class:	CS 6-1	**Semester:**	Spring 2026
Course Code:	CS 3703	**Lab Instructor:**	Junaid Khan
Subject:	Artificial Intelligence		

GitHub Repository: https://github.com/MohmmadSami/AI_Labs.git

1. Learning Objective

The objective of this lab is to implement the classic game of Tic Tac Toe using Artificial Intelligence. Students learn how to build an Evaluation (Heuristic) Function and apply the Minimax Algorithm so that the AI plays optimally against a human opponent.

---

2. Theory

2.1 Evaluation Function

The Evaluation Function (also called a Heuristic Function) is the core of any game-playing AI. It reads the current board state and returns a numeric score that tells the AI how favourable the position is:

- X wins (3 in a row) → Score = +10 (good for Maximizer)
- O wins (3 in a row) → Score = -10 (good for Minimizer)
- No winner / Draw → Score = 0

The function checks all 8 possible winning lines: 3 rows, 3 columns, and 2 diagonals.

2.2 Minimax Algorithm

Minimax is a recursive decision-making algorithm used in two-player zero-sum games. It simulates every possible future move on the game tree:

- X (AI) is the Maximizer — always tries to achieve the highest possible score.
- O (Human) is the Minimizer — always tries to achieve the lowest possible score.
- The AI backtracks (undoes moves) as it explores, keeping the real board unchanged.
- Depth is tracked so the AI prefers quicker wins over slower ones.

Because Tic Tac Toe has a small search space (9! = 362,880 possible games), Minimax can explore the complete game tree in milliseconds, making the AI unbeatable.

3. Implementation

3.1 Key Functions

Function	Description
`evaluate(b)`	Checks all 8 win lines, returns +10 / -10 / 0
`moves_left(b)`	Returns True if any empty cell remains on the board
`minimax(b, depth, is_max)`	Recursively explores all moves, returns optimal score
`best_move(b)`	Calls minimax for every empty cell, returns best (row, col)
`_human() / get_human_move()`	Handles and validates the human player's input
`_check() / get_result()`	Detects win or draw after each move
`_reset() / reset_game()`	Clears the board and starts a new round

3.2 CLI Version — Sample Code

```
def evaluate(b):
    lines = [rows, columns, diagonals]  # 8 win lines
    for line in lines:
        if line[0] == line[1] == line[2] != '_':
            return 10 if line[0] == 'x' else -10
    return 0   # No winner
```

```
def minimax(b, depth, is_max):
    score = evaluate(b)
    if score == 10:  return score - depth  # X wins
    if score == -10: return score + depth  # O wins
    if not moves_left(b): return 0         # Draw
    # Try every empty cell, keep best score, undo move
```

4. Output

The following screenshots show the running output of both the CLI and GUI versions of the Tic Tac Toe AI program.

4.1 CLI Version Output

Run command: python3 tic_tac_toe_cli.py

[CLI Version Screenshot — Insert Screenshot Here]

```
sami@sami-Latitude-E6440:~/Downloads$ python3 tic tac toe cli.py
==========================================
   TIC TAC TOE  —  AI Lab 06
   Riphah University
==========================================
  You play O (Minimizer)
  AI  plays X (Maximizer) using Minimax

==========================================
   NEW GAME — You = O   |   AI = X
==========================================
  Tip: Enter row and column (0–2) to place O

     0   1   2
   +---+---+---+
 0 |   |   |   |
   +---+---+---+
 1 |   |   |   |
   +---+---+---+
 2 |   |   |   |
   +---+---+---+

  Your turn (O):
  Enter your move (row col) e.g. 1 2 : 2 2

     0   1   2
   +---+---+---+
 0 |   |   |   |
   +---+---+---+
 1 |   |   |   |
   +---+---+---+
 2 |   |   | O |
   +---+---+---+

  AI (X) is thinking...
  AI placed X at row=1, col=1

     0   1   2
   +---+---+---+
 0 |   |   |   |
   +---+---+---+
 1 |   | X |   |
   +---+---+---+
 2 |   |   | O |
   +---+---+---+
```

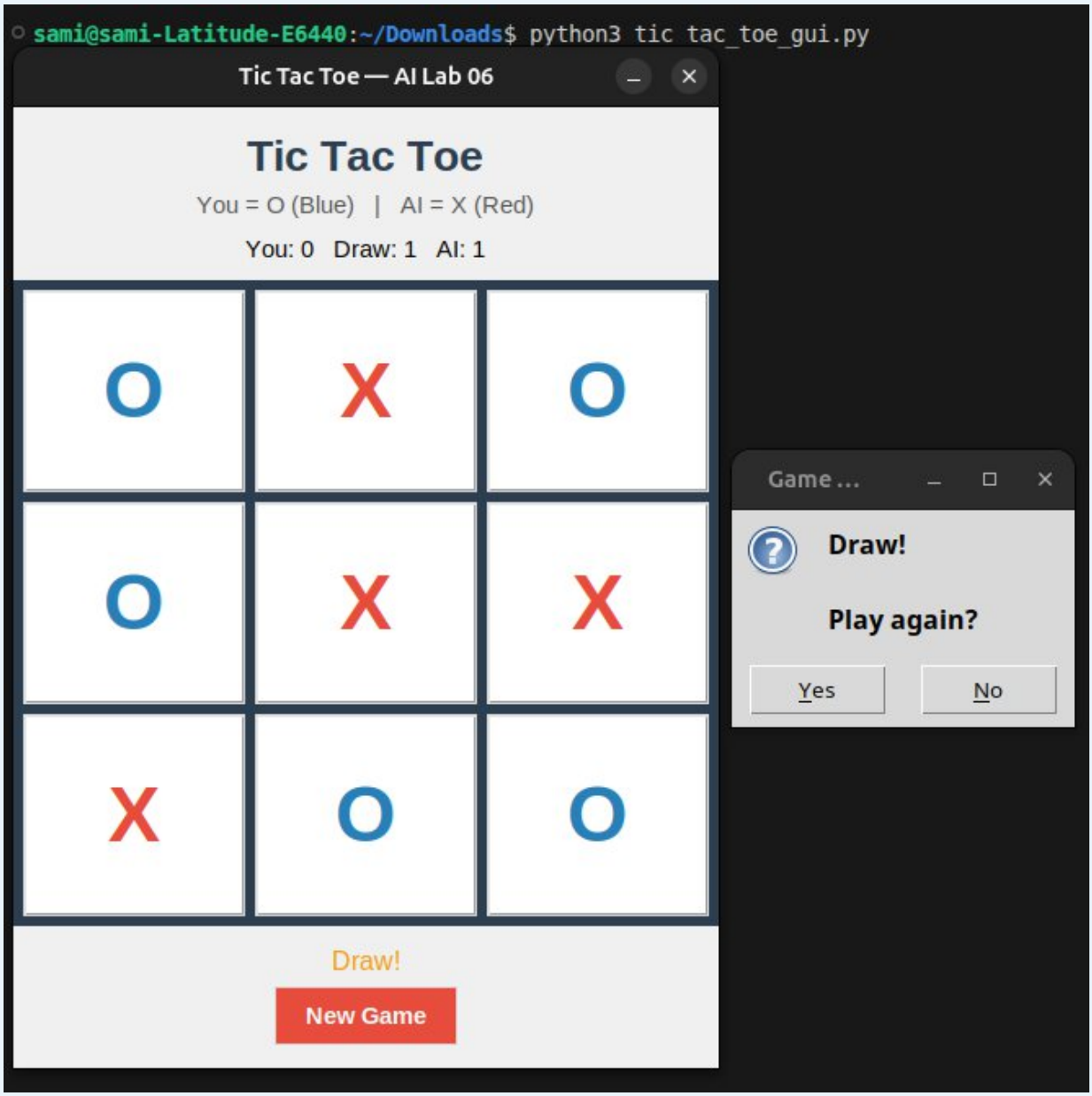
```
 Your turn (O):
 Enter your move (row col) e.g. 1 2 : 1 2

    0   1   2
  +---+---+---+
0 |   |   |   |
  +---+---+---+
1 |   | X | O |
  +---+---+---+
2 |   |   | O |
  +---+---+---+

 AI (X) is thinking...
 AI placed X at row=0, col=2

    0   1   2
  +---+---+---+
0 |   |   | X |
  +---+---+---+
1 |   | X | O |
  +---+---+---+
2 |   |   | O |
  +---+---+---+

 Your turn (O):
 Enter your move (row col) e.g. 1 2 : 1 1
 ✗ Cell already taken! Choose another.

 Enter your move (row col) e.g. 1 2 : 0 0

    0   1   2
  +---+---+---+
0 | O |   | X |
  +---+---+---+
1 |   | X | O |
  +---+---+---+
2 |   |   | O |
  +---+---+---+

 AI (X) is thinking...
 AI placed X at row=2, col=0
```

```
    0   1   2
  +---+---+---+
0 | O |   | X |
  +---+---+---+
1 |   | X | O |
  +---+---+---+
2 | X |   | O |
  +---+---+---+

 ★ AI (X) wins! Better luck next time.


 Score → You (O): 0  |  Draw: 0  |  AI (X): 1
 Play again? (y / n): n

 Thanks for playing! Goodbye.

sami@sami-Latitude-E6440:~/Downloads$
```

4.2 GUI Version Output

Run command: python3 tic_tac_toe_simple200.py

[GUI Version Screenshot — Insert Screenshot Here]



5. Conclusion

This lab demonstrated how AI can play Tic Tac Toe perfectly using the Minimax algorithm combined with an Evaluation Function. The Evaluation Function assigns a score to every board state, while Minimax recursively explores all possible future moves to select the optimal one. Both a CLI and a GUI version were implemented using Python, with the GUI built using the tkinter library. The AI is unbeatable — the best a human can achieve is a draw.

GitHub: https://github.com/MohmmadSami/AI_Labs.git

Mohammad Sami | 53199 | CS 6-1 | Spring 2026